

Tourist AI Agent (Hodari) — System Architecture

An agentic tourist assistant for the 2026 FIFA World Cup,
built on Gemini 3, Google Cloud Agent Builder, and MongoDB Atlas.

1. System overview

Hodari is a three-tier system organized around a multi-agent architecture. A web/mobile client captures user intent through chat, voice, and map interaction. A thin orchestrator agent running on Google Cloud Agent Builder coordinates three specialist sub-agents — a **Planner**, a **Researcher**, and an **Itinerary** agent — each powered by Gemini 3 with a tightly scoped prompt and its own subset of tools. The tools read and write through the MongoDB MCP server to a MongoDB Atlas cluster that stores user profiles, a vectorized catalog of places, and the full interaction history. Google Maps Grounding and Google Routes provide the external real-world data the agents ground their answers in.

The client communicates with the orchestrator over HTTPS, with the response stream delivered through Server-Sent Events so the UI updates progressively as the agents reason. The orchestrator communicates with MongoDB through the MCP protocol — this is the partner integration that qualifies the project for the MongoDB prize bucket. Maps Grounding and Routes are called as standard Gemini function-calling tools from inside the Researcher and Itinerary agents respectively, not through MCP.

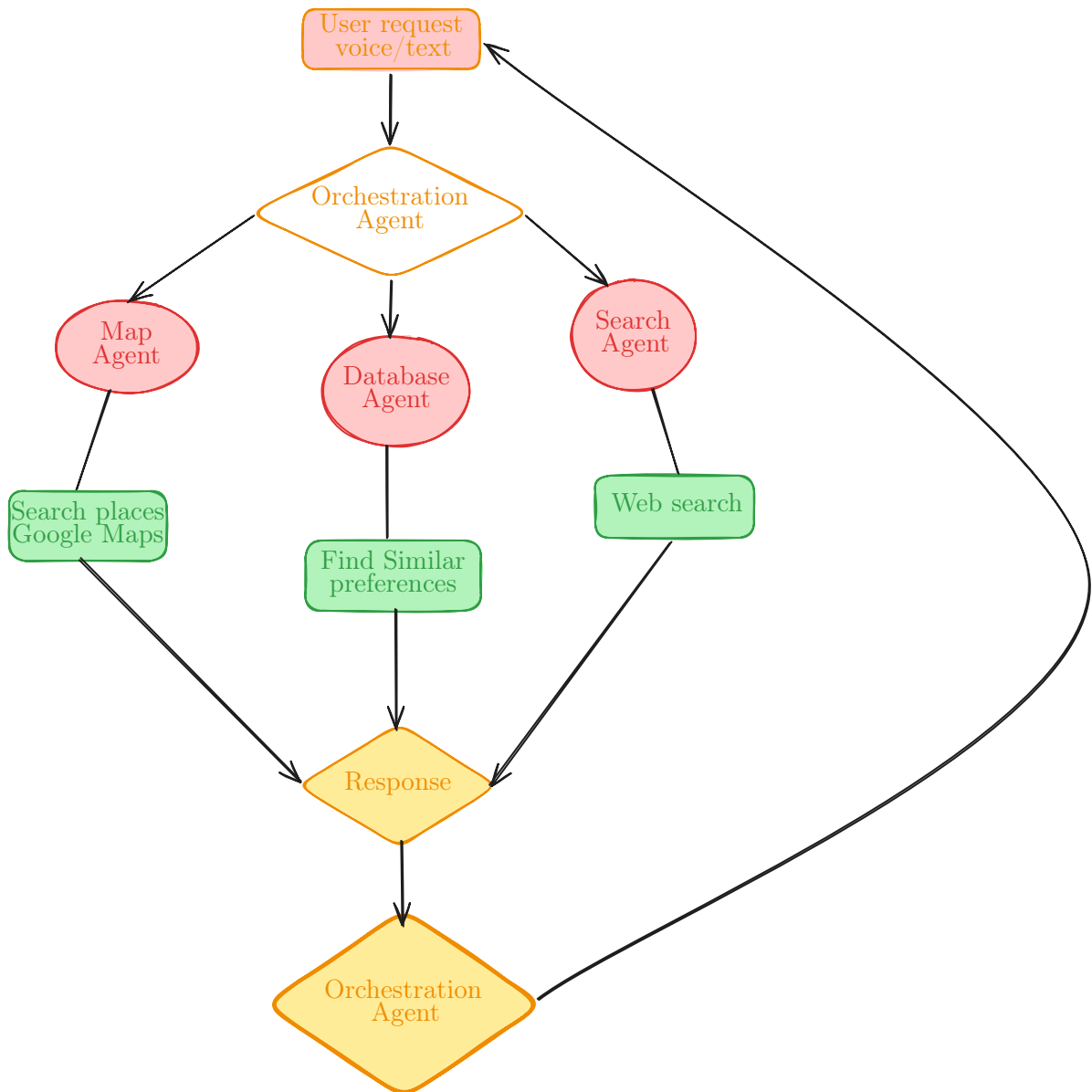
The full deployment runs on Google Cloud Run behind Agent Builder's orchestration layer.

2. Why multi-agent

A single Gemini instance with all four tools is sufficient to solve the problem; the system is split into specialists because each phase of the request lifecycle has a distinctly different reasoning style. Decomposing user intent into a constraint-satisfaction plan, gathering grounded evidence from external APIs, and sequencing that evidence into a feasible itinerary with a coherent narrative are three different jobs that benefit from three different prompts, three different success criteria, and — critically — the ability to run information gathering in parallel.

The cost is roughly double the token spend per turn and additional integration complexity in the contracts between agents. The benefit is faster perceived response time (parallel research), cleaner separation of concerns for debugging, and a design that directly exercises the multi-agent primitives Agent Builder was built for.

The system is deliberately capped at three sub-agents plus an orchestrator. Any further decomposition (separate agents for restaurants vs. attractions vs. nightlife, for example) collapses back into tool-parameter distinctions and does not justify the additional inter-agent latency.



3. Client tier

The client is a single Next.js application that serves both as a responsive web app and as a Progressive Web App installable on mobile. The four primary UI components are:

- a conversational chat panel,
- an interactive map with custom pin styling,
- an itinerary card stack supporting swipe gestures for accept/reject feedback,
- a push-to-talk voice input button backed by the browser's `MediaRecorder` API.

The client holds no business logic. It manages local UI state, streaming response rendering, and audio capture. Every reasoning step happens server-side in the orchestrator and its sub-agents. This keeps the client thin enough that the same codebase serves desktop browsers, mobile browsers, and the installed PWA without divergence.

4. Orchestrator agent

The orchestrator is the entry point for every user turn and the only agent the client communicates with directly. It runs on Google Cloud Agent Builder, deployed as a containerized service on Cloud Run. Its responsibilities are deliberately narrow.

Session state. It owns session state and conversation history. At the start of every turn, the orchestrator loads the user’s profile and recent conversation from MongoDB through the MCP server. At the end of every turn, it persists updates — including any user feedback signals from the previous response — back to the same store. The orchestrator does not perform tourism reasoning itself; that stays in the specialists.

Routing. It routes between sub-agents. For a fresh request, the orchestrator invokes the Planner, then the Researcher, then the Itinerary agent in sequence, passing each one the relevant slice of context. For follow-up turns where the user reacts to a previous itinerary, the orchestrator decides which sub-agents need to re-run: a simple swap might only need the Itinerary agent, a category-level rejection might need both the Researcher and the Itinerary agent, and a fundamental change in intent (“actually, I want to skip lunch and go shopping”) routes back through the Planner.

Memory writes. It owns the `save_preference` tool. Memory writes are session-level operations rather than specialist reasoning, so they belong with the agent that owns session state.

5. Planner agent

The Planner is the first specialist invoked on a fresh request. It receives the user’s message along with the loaded profile and conversation history, and emits a structured plan as JSON. The plan identifies the user’s high-level goal, decomposes it into sub-tasks, and surfaces the constraints the downstream agents must respect: time budget, money budget, dietary flags, accessibility needs, current location, and any negative signals from past interactions.

The Planner does not call any tools. Its job is reasoning about intent, not gathering evidence. This separation keeps its prompt short and its output deterministic enough to be validated with a Pydantic schema on the way out. A malformed plan triggers a single retry with the validation error appended to the prompt; a second failure surfaces as a graceful error to the user rather than poisoning the downstream agents.

The Planner’s structured output is the contract that the rest of the system depends on. Its schema is defined and locked before the Researcher and Itinerary agents are built.

6. Researcher agent

The Researcher executes the plan’s information-gathering sub-tasks. It receives the Planner’s structured output and orchestrates parallel calls to three tools.

search_places_grounded. Takes a natural-language query, a location, a search radius, and a filter dictionary supporting price level, cuisine, open-now status, and accessibility needs. It returns a list of real places with place ID, name, coordinates, rating, price level, hours, photos, and address. The implementation calls the Google Maps Grounding API, which guarantees the agent never recommends a restaurant that doesn’t exist or has been closed for years.

find_similar_preferences. Takes a user ID, an embedding of the current query (generated by Gemini’s `text-embedding-004` model), a list of candidate place IDs from the previous tool, and a `top-K` parameter. It returns those candidates re-ranked by semantic similarity to the user’s historical preferences. The implementation runs a MongoDB Atlas Vector Search aggregation pipeline that combines cosine similarity on the places embedding field with a personalization score weighted by past interaction signals.

web_search. Uses Gemini’s built-in grounding to pull in time-sensitive context — current events, transit disruptions, festival schedules — that Maps doesn’t return. It is invoked only when the Planner’s output flags time sensitivity.

The Researcher’s value over a sequential agent is the **parallel fan-out**: the three tool calls fire concurrently and the agent merges results before returning. This cuts a six-second turn to roughly two and a half seconds, which matters for a live demo more than judges will admit. The agent’s output is an enriched candidate set: roughly five to ten places per sub-task in the plan, each with grounded metadata and a personalization score.

7. Itinerary agent

The Itinerary agent assembles the final user-facing response. It receives the enriched candidate set from the Researcher along with the original plan, and produces a sequenced itinerary with timing, rationale, and a voice-friendly summary.

It owns one tool: **get_route**. This takes an origin, a list of destination coordinates, a travel mode (walking, driving, transit), and a departure time, and returns distance, duration, and a polyline for each leg. The Itinerary agent uses Routes to verify that a proposed sequence actually fits the user’s time budget. If the first proposed sequence overruns, the agent reorders or drops stops and re-checks, up to a small iteration limit, before composing the final response. This is the difference between a hallucinated plan and a grounded one.

The output is a structured itinerary document with stops, transitions, narrative rationale per stop, and a short summary suitable for text-to-speech playback. The orchestrator passes this back to the client.

8. Data tier

MongoDB Atlas hosts a single cluster with three collections. All access from any agent goes through the MongoDB MCP server; no agent has direct database credentials.

users collection. Stores stable profile data: user ID, email, home country, languages, dietary flags, budget tier, accessibility needs, and account creation timestamp. This is populated during onboarding and updated rarely.

places collection. The personalization substrate. Each document represents a place enriched beyond what Google Maps returns: Google Place ID, name, city, GeoJSON location point with a `2dsphere` index for geo queries, categories, price level, a generated description, a 768-dimensional embedding vector, and metadata for hours, photos, rating, and review summary. The embedding field is indexed by Atlas Vector Search using HNSW with cosine similarity. For the MVP we pre-seed roughly 200 places in the demo city; in production this would grow continuously from a background job that ingests new places returned by `search_places_grounded`.

interactions collection. The long-term memory. Every user reaction becomes a document with user ID, place ID, signal (*liked*, *disliked*, *visited*, *skipped*, *booked*), context, session ID, location at the time, and timestamp. Atlas Vector Search joins user history into every retrieval pipeline so that personalization improves over time.

9. End-to-end request flow

A complete turn through the system traces a predictable path. The user speaks or types a request — for the demo, *“I have 4 hours before the game, \$60 budget, vegetarian, no long lines”* — and the client posts it to the orchestrator with the user ID, location, and session ID.

The orchestrator loads context from MongoDB through the MCP server and invokes the Planner. The Planner reasons about intent and emits a structured plan identifying the high-level goal (find a memorable pre-game experience), the constraints (four hours, sixty dollars, vegetarian, no long lines, current location), and three sub-tasks (find vegetarian restaurants, find one nearby attraction, verify the sequence fits the time window).

The orchestrator passes the plan to the Researcher. The Researcher fires `search_places_grounded` for restaurants and `search_places_grounded` for attractions in parallel, alongside

`find_similar_preferences` for each candidate set. Roughly two and a half seconds later, it returns an enriched candidate pool — ten restaurants and five attractions, each scored against the user’s history.

The orchestrator passes the candidates and the plan to the Itinerary agent. The Itinerary agent proposes a three-stop sequence, calls `get_route` to compute walking times between stops, confirms the total fits inside four hours including dwell time at each location, and composes the final response: a structured itinerary, map pins, a per-stop rationale, and a short voice summary.

The orchestrator finalizes the turn by streaming the response back to the client over Server-Sent Events. The map animates new pins in, the card stack populates progressively, and the voice summary plays through text-to-speech if voice was the input mode.

The user swipes left on the second stop — “too touristy.” The client sends this as a new turn with a feedback payload. The orchestrator writes the negative signal through `save_preference`, then makes a routing decision: this is a single-stop substitution, not a fundamental change in intent, so it re-invokes the Researcher (to find replacement candidates with the new negative signal influencing the vector search) and the Itinerary agent (to resequence). The Planner is skipped. A revised itinerary streams back within a few seconds.

The next day, the user flies to the next host city. They open Hodari and ask for breakfast. The orchestrator loads their profile and interaction history — including the “too touristy” signal — and the entire pipeline runs again with the learned preferences influencing the Researcher’s vector search from turn one. No re-onboarding, no repeated mistakes.

10. Inter-agent contracts

The hardest part of a multi-agent system is not the prompts; it is the contracts between agents. Three JSON schemas are defined upfront and validated with Pydantic on every hop.

- The **Planner** emits a **Plan** containing the user’s goal, the constraint set, and an ordered list of sub-tasks.
- The **Researcher** emits a **CandidateSet** containing scored, grounded places grouped by sub-task.

- The **Itinerary agent** emits an **Itinerary** containing a sequence of stops, transitions with travel times, a per-stop rationale, and a voice summary string.

Each schema rejects malformed input with a single retry attempt; a second failure surfaces as a graceful error to the user rather than letting bad data flow through.

This contract discipline is what prevents the silent degradation that kills most multi-agent systems — one agent emits slightly off-spec output, the next agent does its best to interpret it, and three hops later the user gets nonsense.

11. Deployment topology

Four agent services run in production: the orchestrator, Planner, Researcher, and Itinerary agents, each deployed as a Cloud Run service and coordinated by Agent Builder. For MVP, all four can run in a single Cloud Run service to simplify deployment and reduce cold-start cost; production would split them for independent scaling.

The Next.js client is deployed to Vercel or Cloud Run with a CDN in front of it. MongoDB Atlas runs on its own managed infrastructure with the MCP server hosted alongside it. Service-to-service authentication uses Google Cloud IAM for the Cloud Run hops and short-lived API keys for the MCP and external API hops. Secrets live in Google Secret Manager and are mounted as environment variables at container start.

Observability is handled through Cloud Run's built-in logging and tracing, with structured logs emitted at every agent boundary so any user turn can be replayed for debugging. This is essential when the demo video gets recorded — being able to point at a specific turn and walk through what each sub-agent did is also the strongest possible answer to a judge asking how the system actually works.

12. What's deliberately out of scope for the MVP

Booking, payments, email integration, calendar sync, OAuth to third-party accounts, and multilingual support are all listed in the roadmap and shown in the final storyboard panel of the demo, but none of them ship in the hackathon submission. Each represents one to two weeks of work and would compete for time with the core multi-agent loop.

The submission demonstrates that the loop works end-to-end with grounded data, persistent memory, and live replanning. Production extension is straightforward — each additional capability is either a new tool registered with an existing sub-agent or, if it has a genuinely distinct reasoning style, a new specialist sub-agent invoked by the orchestrator.